

RAG-Forge: Building an Evaluation-Driven RAG System with Span-Anchored Synthetic Evaluation on a Fully Local Stack

Technical report covering Phases 1–4: infrastructure, ingestion, baseline dense RAG, evaluation dataset and retrieval benchmark

Nabeel Ahmad

RAG-Forge project · September 2026 · work in progress · repository commits f2854e3 → f46239c

Abstract. Retrieval-augmented generation (RAG) systems are commonly judged by inspecting a handful of answers. We describe the first four phases of RAG-Forge, a system built so that every technique can be switched on or off and measured against a baseline. The system runs entirely on a laptop-class machine (Apple M1, 8 GB) using open models: bge-small-en-v1.5 embeddings, PostgreSQL with pgvector, and Qwen2.5-3B-Instruct served locally through MLX. We ingest a pinned snapshot of the Kubernetes documentation (551 documents, 3,855 chunks), resolving the Hugo template syntax that would otherwise pollute both lexical and dense indexes, and we chunk with a structure-aware parent/child scheme whose design choices were each driven by measurements on the real corpus. Auditing the chunker against the embedder's actual tokenizer exposed silent truncation of 4% of embedding inputs, which a calibrated token estimator removed. For evaluation we generate 316 synthetic questions across nine types (including multi-hop over real cross-document links, code-constructed false premises and corpus-verified unanswerable questions). Every item passes deterministic validators (only 17% of 1,901 generations were kept), and relevance is anchored to exact character spans rather than chunk identifiers, so labels survive re-chunking. We introduce Evidence Recall@K for multi-span questions and report a dense-retrieval baseline on the held-out test split: Evidence Recall@5 of 0.701 (95% bootstrap CI 0.641–0.759) and MRR of 0.624 (0.573–0.682). The baseline is weakest on ambiguous (0.47) and comparison (0.45) questions, pointing directly at query rewriting and reranking, the next techniques to be evaluated. We document the failure modes found at each step, including those of the evaluation data itself.

1 Introduction

Most RAG prototypes implement a single path, *query* → *vector search* → *LLM* → *answer*, and judge quality by eye. That makes it impossible to answer the questions that matter when engineering such a system. Does hybrid retrieval beat dense retrieval on this corpus, and for which kinds of questions? How much does a cross-encoder reranker improve early precision, and at what latency? When corrective retrieval fires, does the extra LLM call pay for itself? RAG-Forge is organised around these questions. Each technique (BM25, hybrid fusion, reranking, parent-child retrieval, query rewriting, corrective RAG [2], a Self-RAG-inspired loop [3] and claim-level verification) is to be implemented behind a configuration switch and benchmarked against the same baseline, on the same held-out questions, with quality, latency and cost reported together.

This report covers the first four of eighteen planned phases: the infrastructure, the ingestion pipeline, a baseline dense RAG pipeline, and the evaluation dataset and retrieval benchmark on which all later comparisons will rest. The later techniques are *not* implemented yet, and no results are reported for them. We describe the architecture and every design choice, and we give particular attention to the evaluation dataset: how each question type is generated, how items are validated, and where the data is still weak.

Contributions of this phase

- **A span-anchored evaluation design.** Relevance labels are exact character spans in the stored document text. Chunk-level relevance is recomputed against whatever index is being evaluated, so chunking itself becomes a benchmarkable variable instead of something that silently invalidates the labels (§6.1).

- **A validated synthetic dataset built with a small local model.** 316 items across nine question types; 17% of generator outputs survive deterministic validation; three rounds of manual pilot review each changed the generator design (§6).
- **Evidence Recall@K** for questions that need several pieces of evidence, with bootstrap confidence intervals and per-type breakdowns (§7).
- **Measurement-driven ingestion.** Corpus-specific template resolution, and a chunker whose parameters were corrected by audits against the real tokenizer and the real corpus (§4).
- **A fully local, free and reproducible stack,** with a provider abstraction that allows the same experiments to be re-run with a stronger model later (§3, §5).

2 Background and related work

RAG. Retrieval-augmented generation conditions a language model on retrieved passages [1]. Retrieval can be lexical, typically BM25 [4], dense, using bi-encoder embeddings such as the BGE family [8] indexed with approximate nearest-neighbour structures such as HNSW [7], or hybrid, commonly fused by Reciprocal Rank Fusion [5]. Cross-encoders, and sequence-to-sequence rankers such as monoT5 [11], rerank a candidate list at higher cost.

Adaptive RAG. Corrective RAG (CRAG) [2] adds a lightweight retrieval evaluator, a fine-tuned T5-large in the original work, that grades retrieved documents and triggers corrective actions when they are judged irrelevant. Self-RAG [3] fine-tunes a generator (Llama-2 7B/13B) to emit reflection tokens that decide whether to retrieve and whether the output is supported. RAG-Forge will implement *practical adaptations* of these ideas and does not claim to reproduce either paper. In particular, its Self-RAG-style decisions will be made by separate graders rather than by a model trained with reflection tokens.

Evaluation. Retrieval is conventionally scored with Recall@K, Precision@K, MRR and NDCG [6]. RAG-specific frameworks such as RAGAS [9] and ARES [15] use LLMs to judge faithfulness and relevance, and ARES also generates synthetic queries for evaluation. LLM judges are known to have biases, including self-preference [14]. Small models trained for grounding verification, such as MiniCheck [10], offer an alternative to large judges for claim-level checks. Benchmarks built from passages are known to interact with retrieval method; BEIR [16], for example, found BM25 to be a strong zero-shot baseline. We return to this lexical-bias concern for our own synthetic questions in §6.7.

3 System architecture

RAG-Forge is a single Python service rather than a set of microservices. The spec explicitly prioritised correctness, evaluation and reproducibility over deployment complexity. Figure 1 shows the components as of Phase 4.

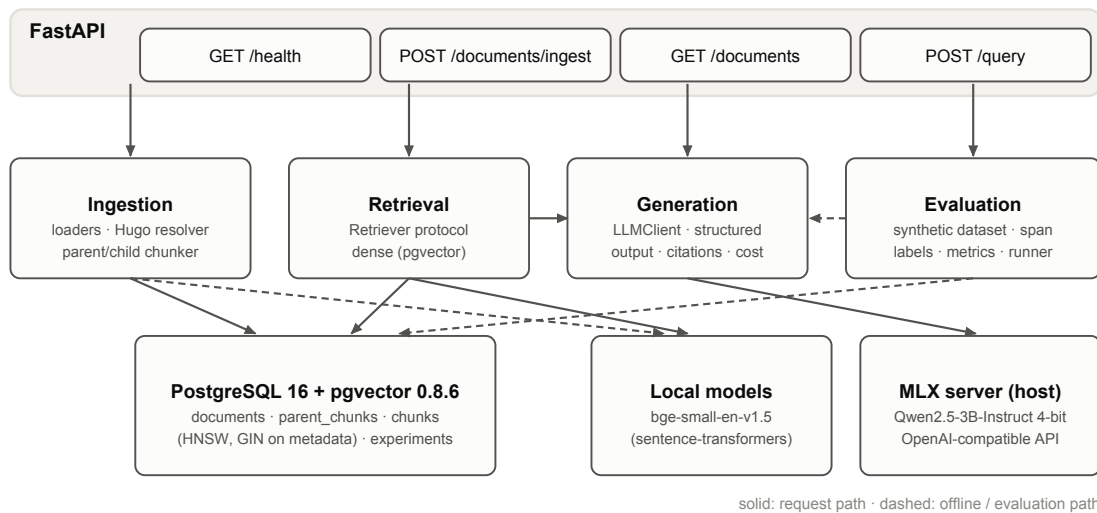


Figure 1. Component architecture after Phase 4. The API and the evaluation scripts share one code base. The LLM runs on the host because MLX needs the Apple GPU, which containers cannot access. The API container reaches it through `host.docker.internal`.

3.1 Technology choices

Table 1. Core technology choices and their rationale.

Concern	Choice	Reason
API	FastAPI, Pydantic v2	Typed request and response models; dependency injection makes every component replaceable in tests
Storage	PostgreSQL 16 + pgvector 0.8.6	Vectors, metadata and document text in one transactional store; metadata filters are plain SQL; no second system to keep in sync
ANN index	HNSW with operator class matched to the configured metric	Standard, and representative of larger corpora even though ~4k vectors could be searched exactly
DB access	Synchronous SQLAlchemy 2.0 + psycopg 3	Expensive work (embedding, reranking) is CPU/GPU-bound, so async I/O adds complexity with little benefit at this scale
Embeddings	BAAI/bge-small-en-v1.5 (384-d, 512-token limit)	Runs locally beside the LLM within 8 GB of RAM
LLM	Qwen2.5-3B-Instruct [12], 4-bit, served by mlx-lm 0.31.3	Free, offline, uses the M1 GPU; its OpenAI-compatible server needs no custom client
Packaging	Docker Compose (API + Postgres); host port 5434	Ports 5432 and 5433 were occupied on the development machine

3.2 Configuration as the experiment interface

Every tunable value lives in one validated settings object (pydantic-settings), read from flat environment variables (`TOP_K`, `RELEVANCE_THRESHOLD`, ...) and grouped internally into database, embedding, chunking, retrieval, adaptive, LLM and observability sections. The design rules are:

- **No hard-coded thresholds.** Components receive settings explicitly, which is what allows the experiment runner to build a different configuration per experiment.
- **Validation at load time.** This covers ranges (thresholds in $[0, 1]$), hard caps on loop iterations (≤ 5 , so a typo cannot create a runaway corrective loop) and cross-field constraints ($\text{RERANK_TOP_K} \leq \text{RERANK_CANDIDATES} \leq \text{RETRIEVAL_CANDIDATES}$; fusion weights not both zero).
- **Lazy secret checks.** API keys are validated only when a provider client is built, so tests and offline tools run without secrets.
- **Secret-free snapshots.** `Settings.snapshot()` produces a copy of the configuration with secrets removed, stored with every experiment report.

3.3 Data model

The schema is `documents` → `parent_chunks` → `chunks`, plus an `experiments` table. Three decisions shape everything downstream:

- 1 **Chunks are offsets, not copies.** Each document stores its full normalised text (`documents.content`). Every parent and child chunk is exactly `content[start_char:end_char]`. Evaluation labels refer to these offsets.
- 2 **Deterministic identifiers.** Document IDs are hashes of the normalised source path, so re-ingestion is an idempotent upsert and dataset references stay stable.
- 3 **Fixed embedding dimension and model isolation.** The vector column has a fixed dimension, and every chunk records the embedding model that produced its vector. Retrieval only ranks vectors from the current model.

3.4 Engineering practice

The work was committed phase by phase, with each phase tested before its commit. At commit `f46239c` the suite has 144 tests:

- unit tests for configuration, parsing, chunking invariants, the HTTP client's retry policy, structured output, citations and metric definitions
- integration tests against a dedicated PostgreSQL test database
- evaluation tests for dataset validation, span labelling and the evaluation runner

Metric functions are checked against hand-computed values. Integration tests run against real pgvector, not mocks.

4 Corpus and ingestion

4.1 Corpus

We use the English Kubernetes documentation from the `kubernetes/website` repository (CC BY 4.0), pinned to commit `5dd61e1` of 2026-09-23 (docs version v1.37). We take the `concepts/`, `tasks/` and `reference/glossary/` subtrees: 570 Markdown files, 551 of which contain prose (19 are empty section-index pages). A committed manifest records a SHA-256 for every file, so drift is detectable. The corpus was chosen because it supports factual, configuration, troubleshooting, comparison and multi-hop questions, and because its heavy cross-linking can be used to construct multi-hop pairs (§6.3).

4.2 Template resolution

The documentation is written for the Hugo site generator and is dense with template shortcodes. In `concepts/` and `tasks/` alone we counted 744 `glossary_tooltip`, 585 `note`, 354 `feature-state`, 277 `code_sample` and 187 `skew` shortcodes. Indexed raw, BM25 and the embedder would match template syntax instead of prose. A corpus-specific resolver approximates what a reader sees on `kubernetes.io`:

- tooltips become their text, or the glossary term's title
- glossary definitions are expanded from the glossary files, recursively, with a depth limit
- version shortcodes become the pinned version
- admonitions become an inline “Note:” label
- boilerplate includes are dropped

Seven rare shortcodes that the first pass did not handle were found by logging unknown names. Two of them (`link`, `relref`) were deleting visible text. After resolution, 0 of 3,855 chunks contain template syntax. Each document stores its canonical `kubernetes.io` URL for citation.

4.3 Structure-aware parent/child chunking

The chunker only chooses boundaries; it never rewrites text. It works in three steps:

- 1 The text is cut into *blocks*: paragraphs, with fenced code blocks kept whole and headings attached to the block that follows them.
- 2 Oversized blocks are split into sentences, or lines for code, then into word windows, then into character slices. The size limits are therefore strict.
- 3 *Parents* are consecutive heading sections merged up to `PARENT_CHUNK_SIZE`. *Children* are packed greedily inside each parent, with sentence-aligned overlap.

Four invariants are tested on synthetic documents and on a sample of the real corpus:

- no non-whitespace character is lost
- every child lies inside its parent
- no chunk exceeds its limit
- the output is deterministic

Three parameters of this design were changed because measurements on the corpus showed a problem (Table 2).

Table 2. Chunking changes driven by corpus measurements.

Observation (measured on the corpus)	Change	Effect
Parents were not allowed to span top-level (H2) sections. Kubernetes pages have many short H2 sections, so 50.1% of children were identical to their parent, and parent-child retrieval would add no context	Allow merging across H2 boundaries (never across PDF pages)	Children identical to their parent: 50.1% → 8.0% (6.3% with final settings); median parent/child size ratio 1.0 → 3.8×; chunks under 30 tokens: 195 → 10
The source is hard-wrapped, and treating every newline as a sentence end produced chunks that began mid-sentence	Split only after . ! ?, after a line-ending colon, or before list items	Chunks starting in lowercase (a proxy for mid-sentence starts): 31 of 3,062 (1%), all of them legitimate on inspection
A word-count token estimate counted a 5,000-character base64 run as one token	Cost long runs by length; slice oversized unsplittable runs	Hard size limits; see the calibration in Table 3

4.4 Calibrating the token estimate against the real tokenizer

Chunk sizes are configured in tokens, but the chunker deliberately uses a model-independent estimate, so that changing the embedding model does not move chunk boundaries. An audit script compares the estimate with bge-small's WordPiece tokenizer on every stored chunk and counts embedding inputs above the model's 512-token limit. The first estimator undercounted, and **123 of 3,062 embedding inputs (4%) were being silently truncated** by the embedder. The worst cases were hex container IDs in command output, wide Markdown tables and long CamelCase feature-gate names.

The calibrated rules are:

- word pieces cost one token per ~7 characters
- runs that mix letters and digits cost one per ~2 characters
- CamelCase identifiers are costed per component

The child size was also reduced from 400 to 360 estimated tokens.

Table 3. Token-estimate calibration against the bge-small tokenizer (whole corpus). Overestimates are rarer and harmless: only 4 chunks are overestimated by more than 1.5×. The extreme case (119×) is a base64 certificate, which WordPiece maps to a single [UNK] token.

	Naive estimator, 400-token children	Calibrated estimator, 360-token children
Estimate / actual, median	0.874	0.978
Estimate / actual, 5th–95th percentile	0.770 – 0.956	0.862 – 1.086
Worst underestimate	0.43	0.70
Largest embedding input (real tokens)	746	491
Embedding inputs truncated at 512 tokens	123 of 3,062 (4.0%)	0 of 3,855

The pipeline also flags any chunk whose embedding input still exceeds the model limit (`embedding_truncated` in chunk metadata), so truncation can no longer be silent.

4.5 Resulting index

Table 4. Index statistics. There are 1,044 parents (median 1,126 tokens), and 0 offset mismatches between stored chunk text and `content[start:end]`. Embedding all 3,855 chunks took 2.7 min on the M1 GPU (MPS) versus ~4 min on the CPU (43 vs 79 ms per chunk).

Category	Documents	Child chunks	Avg child tokens
concepts	185	2,099	289
tasks	204	1,593	292
reference (glossary)	162	163	101
Total	551	3,855	–

Each chunk is embedded as "`<title> > <section path>\n\n<text>`". The stored text is unchanged; the prefix disambiguates chunks such as “Set the field to `true` to ...”. Re-ingestion is idempotent. Each document stores a fingerprint of its content hash, the chunker version and configuration, the embedding model and the context flag, and only documents whose fingerprint changes are rebuilt.

5 Baseline RAG pipeline

5.1 Dense retrieval

All retrievers implement one method, `retrieve(query, k, filters)`, and return a standard result (document ID, chunk ID, text, score, rank, offsets and metadata). The pgvector retriever handles three details that silently change results if ignored:

- **`hnsw.ef_search` caps the number of results.** Its default of 40 would quietly truncate a top-50 candidate list and bias every later retrieve-then-rerank experiment. It is raised per query to `max(40, 2k)`. Verified: `k = 60` returns 60 results.
- **Filtered HNSW search can return fewer than `k` rows.** Filtered queries enable pgvector 0.8's iterative scans and re-sort the results.
- **Rows are restricted to the current embedding model**, so vectors from a previous model are never ranked against new query vectors.

Query latency is ~25 ms once the model is loaded. The first query after process start paid ~12 s of model loading, so the API preloads the embedder at startup.

5.2 Provider-neutral LLM layer

Pipelines depend on a single `complete(messages)` interface. One HTTP client covers every OpenAI-compatible server: local MLX, Ollama, vLLM, or hosted APIs. Its retry policy is explicit:

- timeouts, connection errors and 5xx responses are retried with jittered exponential backoff
- 429 responses are retried, honouring `Retry-After`

- other 4xx responses fail immediately
- the number of attempts is bounded

Structured output does not rely on server-side JSON enforcement, which `mlx_lm.server` lacks:

- the schema is described in the prompt
- replies are parsed tolerantly (fences stripped; the first balanced object extracted)
- the validation error is fed back for one repair attempt
- otherwise a `StructuredOutputError` is raised; no default is substituted silently

Every call is recorded with its purpose, token counts and latency. Models without a configured price are reported as *unpriced* rather than counted as free, so a paid model with a missing price cannot make a strategy look cheap.

5.3 Answer generation and citations

The baseline embeds the query, retrieves the top-k chunks, and makes exactly one LLM call over numbered sources. The model must cite inline (`[n]`) and use an exact abstention phrase when the sources lack the answer, so abstention is detected deterministically, without a judge. Empty retrieval abstains without calling the LLM. Citation numbers are mapped back to chunk, section, page, URL and character span; numbers that match no source are reported as *invalid citations*.

With the 3B model, the first prompt (all rules in the system message) produced valid citations in only 1 of 8 answers on a sanity probe. Adding a format example raised this to 2 of 8. Restating the citation rule after the sources raised it to **8 of 8**. Small models tend to lose system-prompt instructions behind ~1.5k tokens of context. This is an 8-question probe used to choose a reasonable baseline prompt, not a benchmark result.

Table 5. Observed behaviour of the local model. These are sanity checks, not benchmarks.

Measurement (dev machine)	Value
Structured grading probe: valid JSON matching the schema	6 / 6; ~1.1 s per call when warm
Grading judgment: a passage stating the answer ("graceful within 30 seconds") marked <code>contains_answer</code>	false on 3 / 3 repeats
Baseline answer generation, top-5 contexts (~1.6–1.8k prompt tokens)	15–19 s
End to end through Docker: retrieval / generation	0.14 s / 19.1 s; 1,625 input and 17 output tokens; \$0.00
Unanswerable probe ("What is the capital of France?")	abstained correctly

The grading failure in Table 5 is important for later phases. The 3B model formats output reliably but judges relevance poorly. Following CRAG's own design of a small dedicated evaluator [2], relevance grading will use a cross-encoder (or monoT5 [11]), and claim verification will use MiniCheck [10]. The LLM grader remains a switchable comparison.

6 The evaluation dataset

The dataset, `kubernetes_v1`, contains 316 items and is **synthetic**. Questions, answers and evidence selections were produced by the local 3B model and filtered by deterministic validators. It is **not human-annotated**, and no item is labelled as human-reviewed.

6.1 Design principles

Span-anchored relevance

Each item's evidence is a list of exact spans (`document_id`, `start_char`, `end_char`, `quote`) into `documents.content`. At evaluation time a chunk is relevant to a span if it covers at least 50% of the span's characters, computed against the index being evaluated. The stored `relevant_chunk_ids` are only a convenience snapshot. As a result, the same dataset can evaluate different chunk sizes, overlaps or parent-child settings. A span that no current chunk covers is reported and counted as a miss.

Held-out test split

Items are split 30% dev and 70% test, stratified by question type with a fixed seed (97 dev, 219 test). Thresholds and weights may be tuned only on dev; reported results use test. Tuning on the reported questions would overstate every tuned technique. With about 100 dev items, k-fold cross-validation over the dev split is available for threshold tuning.

Provenance

Every item records:

- the generator model
- the prompt version
- the list of validators it passed

The companion metadata file records the corpus commit, chunker fingerprint, seeds, per-type counts, every generation run, and per-type rejection counts for each validator.

6.2 Generation procedure

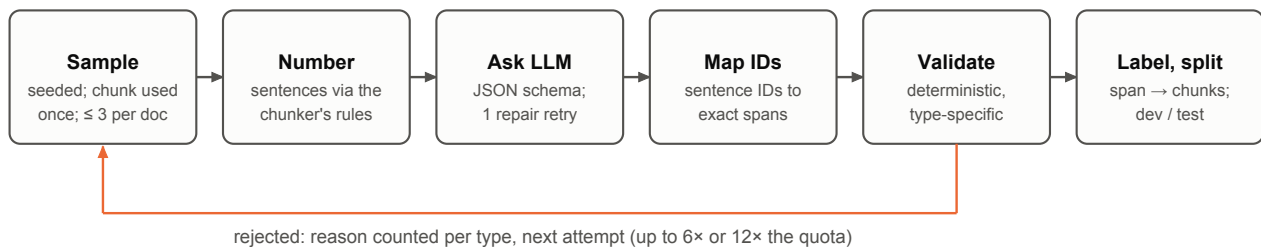


Figure 2. Per-item generation loop. Passages are shown to the model as numbered sentences, using the chunker's own sentence rules, and the model returns sentence numbers, not quotes. Evidence spans are then mapped deterministically, so they are always exact.

Sampling is seeded. Each chunk is used at most once, and each document yields at most three items. Chunks that are mostly code, or more than 5% HTML markup, are skipped. The items that pass are joined with dataset-wide checks:

- the question is well formed and self-contained: it may not say “the passage”, “the example” or “in this case”
- the question is not a near-duplicate of an earlier one (content-word Jaccard below 0.8)
- the answer is non-empty and *supported*: at least 50% of the stemmed content words it adds beyond the question must appear in the evidence
- the evidence maps to at least one indexed chunk

6.3 Question types

Table 6 summarises how each of the nine types is built and which type-specific validators it must pass. The notes below explain the less obvious ones.

Table 6. Construction and validation per question type.

Type	Construction	Type-specific validation
simple_factual	One chunk → direct factual question	No 8-gram copied from the evidence
semantic	One chunk → question in different words	At most 40% of the question's stemmed, non-generic content words appear in the evidence
keyword_heavy	One chunk plus a required identifier (flag, dotted field, camelCase)	The identifier appears in both the question and the evidence
numerical	Chunk containing number + unit	The answer's number appears in the evidence
multi_hop	Chunk A, plus the best-matching chunk of a page that A links to	Answer uses ≥ 2 words unique to each source; neither evidence is only a link; prose only

Table 6. Construction and validation per question type.

Type	Construction	Type-specific validation
comparison	Intro chunks of the two most similar concept pages in one section	As multi-hop
ambiguous	Specific question → vague rewrite (“What happens when it is deleted?”)	The rewrite names neither the page title nor the capitalised subjects; the specific form is kept
adversarial	LLM states a true numeric fact; code changes the number	The fact is $\geq 70\%$ supported by its evidence and shares its number; the true and false values are recorded
unanswerable	(a) invented Kubernetes feature; (b) question about another technology	(a) the invented term appears nowhere in the corpus (SQL); (b) the topic keyword appears nowhere in the corpus, and there are no Kubernetes terms

Multi-hop pairs follow real links

Of the eligible concept and task chunks, 1,244 link to another document in the corpus. For a chunk A containing a link, we take the text around the link and use it as a dense query *restricted to the linked page* (a metadata filter on its URL). The best match is passage B. The pair is therefore connected by the documentation's own structure rather than chosen at random.

Comparisons pair similar pages

A concept page's introduction is paired with the most similar other introduction in the same documentation section, for example ReplicationController with ReplicaSet.

False premises are built by code

The model only states a true fact containing a number. Code then replaces that number with a different value that does not occur in the evidence. The question takes the form “I read that <false statement>. Why is that?”, and the ground truth is the verified true statement.

Unanswerable items are verified against the corpus

A fake feature or an out-of-domain topic is accepted only if a SQL search of all document text finds no occurrence of its key term. Of 20 candidate technologies, only 9 were absent from the corpus; Nginx and Redis, for example, appear in Kubernetes examples.

6.4 How pilot reviews changed the generator

Three pilots of three items per type were read item by item before the full run. Each review exposed failures that the validators of the time did not catch (Table 7). These reviews are the main reason the final validators exist.

Table 7. Failures found by reading generated items, and the resulting design changes.

Pilot	Finding	Design change
1	All 3 adversarial items were wrong. The model wrote “false” premises that were true, or restated the passage	False premises constructed by code from a verified true fact
1	Comparisons of unrelated pages (metrics vs version emulation)	Pair each page with its most similar intro in the same section
1	A “multi-hop” item answerable from A alone; B's evidence was “See [link] for more”	Uses-both-sources check; link-only evidence rejected
1	Semantic questions copied the passage, but Jaccard stayed low because quotes were long	Containment instead of Jaccard
2	A fabricated fact: “the default Pod grace period is 30 seconds” was attached to evidence about CoreDNS caching for 30 seconds. The model had copied the prompt's example sentence	Example removed; answer-support check added
2	Multi-hop: 0 of 3 accepted; 14 of 18 attempts failed because quotes were not copied verbatim	Numbered sentences and sentence-ID evidence instead of quotes
2	Generic words (“Kubernetes cluster”) diluted the paraphrase ratio; “require” did not match “requires”	Light stemming; generic words excluded; threshold 0.4

Table 7. Failures found by reading generated items, and the resulting design changes.

Pilot	Finding	Design change
3	Large YAML blocks, counted as one “sentence”, satisfied the uses-both check by word count	Prose-only passages for two-source questions
3	Evidence that was raw HTML table markup	Skip chunks with > 5% HTML
Full run	At temperature 0, one prompt per topic gives one question: 81 near-duplicates for 9 topics	Topic × angle prompts for out-of-domain questions
Full run	A fake <code>--force-delete</code> flag sits next to the real <code>--force</code> . “Not documented; use <code>--force</code> ” is a correct answer, but the label demanded a flat abstention	Ground truth for nonexistent features changed to accept a correction

6.5 Resulting dataset

The full run (seed 42) produced 275 items. A top-up run (seed 43, extend mode) marked all existing chunks and questions as used and added 41 items to the types that had fallen short.

Table 8. Dataset composition and generation statistics (version c6009ef85e53). *Comparison fell short of its target of 30; the shortfall is reported rather than filled by relaxing validation. The items cover 208 of the 551 documents.

Type	Items	Test	Dev	Accepted / attempts	Rate	Most common rejection
simple_factual	60	42	18	60 / 100	60%	not self-contained (15)
semantic	45	31	14	45 / 423	11%	not paraphrased (304)
keyword_heavy	45	31	14	45 / 251	18%	identifier missing (150)
numerical	30	21	9	30 / 59	51%	number not in evidence (12)
multi_hop	37	26	11	37 / 432	9%	does not use both sources (262)
comparison	19*	13	6	19 / 336	6%	does not use both sources (158)
ambiguous	25	17	8	25 / 79	32%	rewrite names the subject (25)
adversarial	25	17	8	25 / 103	24%	stated fact had no number (40)
unanswerable	30	21	9	30 / 118	25%	near-duplicate (81)
Total	316	219	97	316 / 1,901	17%	

6.6 Examples

One real item per type, taken verbatim from the dataset (answers abridged where marked with ...):

Table 9. Example items (dataset kubernetes_v1).

Type	Question	Ground truth
simple_factual	What role is assigned by default to an extension apiserver to allow it to retrieve a configmap?	The default role named <code>'extension-apiserver-authentication-reader'</code> in the <code>'kube-system'</code> namespace can be assigned.
semantic	What are the common prefixes used for different subsystems in the <code>sysctl</code> interface?	... <code>'kernel.'</code> for kernel parameters, <code>'net.'</code> for networking parameters, and <code>'vm.'</code> for virtual memory...
keyword_heavy	What operators are not available for podAffinity?	The <code>'Gt'</code> and <code>'Lt'</code> operators are not available for podAffinity.
numerical	How many seconds does the container need to be alive before the liveness probe starts failing?	10
multi_hop	What actions are recommended for cleaning up a ConfigMap after it has been changed, and how do these actions relate to the process of recreating existing Pods?	Recreating existing Pods that maintain a mount point to the deleted ConfigMap, and removing the <code>'kustomization.yaml'</code> file...
comparison	How do the methods for protecting secret data differ between Linux and Windows nodes?	On Linux nodes, secret data is stored in memory-backed volumes using a <code>tmpfs</code> filesystem, while on Windows nodes, secret data is written out in clear text...

Table 9. Example items (dataset kubernetes_v1).

Type	Question	Ground truth
ambiguous	What command do you use to see the rollout history of it?	kubectl rollout history deployment/nginx-deployment
adversarial	I read that the Kubernetes project recommends rotating the KEK at least every 180 days to avoid issues related to the number of writes performed with the DEK. Why is that?	That premise is incorrect: ... at least every 90 days ... (true value 90, false value 180)
unanswerable	How can I use the --auto-heal-pods flag to automatically heal pods during the restoration of an etcd cluster?	The corpus does not document '--auto-heal-pods'. A correct response says so (and may point to a related documented feature)...

6.7 Quality audit and known weaknesses

After each run, a random sample was read in full during development by an automated LLM-based reviewer. This is a sanity check, not human annotation. Of 33 sampled items, 25 were acceptable. That count includes one unanswerable item whose label was wrong and has since been corrected. The other 8 were flawed:

- 3 multi-hop items with vague answers, or answers that only partly use the second source
- 1 comparison with a fabricated contrast (“scheduler vs X.509 certificates”)
- 1 numerical item that took “8” from “8 hours late” and dropped the unit
- 1 ambiguous item whose answer misses the question
- 2 simple items with garbled grammar

One further error found while preparing this report shows the limit of word-overlap support checks. An adversarial item attributes `progressDeadlineSeconds` to `.spec.strategy.rollingUpdate`; its value, 600, is correct, but the field actually lives at `.spec.progressDeadlineSeconds`. The sample is too small to estimate per-type error rates. A script exports a stratified sample for human review; only verdicts entered by a person should be reported as human-validated accuracy. The known weaknesses are:

- **Lexical bias.** Questions are written from the passage they are evaluated against, and so share its vocabulary. This favours lexical retrieval, a known property of passage-derived benchmarks [16]. The semantic type only partly counters it.
- **Same model family.** The generator is also the default answer model. This does not affect retrieval metrics, but generation metrics must account for it [14].
- **Uneven label quality.** Multi-hop and comparison items have correct evidence spans more often than good reference answers: their retrieval labels are more reliable than their answer labels.
- **Example-specific questions** (“How many replicas are initially scheduled for the Deployment 'nginx'?”) are answerable but less natural than real queries.
- **Coverage and size.** 208 of 551 documents; 13–42 test items per type.

7 Retrieval metrics

All metrics use binary relevance over a single ranked list per query, computed once at the deepest K . Let R be the set of relevant chunks and $top\text{-}K$ the first K retrieved chunks:

Table 10. Metric definitions. All are unit-tested against hand-computed values.

Metric	Definition
Recall@K	$ R \cap \text{top-K} / R $
Precision@K	$ R \cap \text{top-K} / K$
Hit@K	1 if $R \cap \text{top-K}$ is non-empty, else 0
MRR	1 / rank of the first relevant chunk (0 if none within the deepest K), averaged over queries
NDCG@K [6]	DCG@K / IDCG@K, with $\text{DCG} = \sum \text{rel}_i / \log_2(i+1)$ over binary gains; duplicate hits earn no extra gain; IDCG places $\min(R , K)$ relevant items first
Evidence Recall@K	Fraction of the item's evidence spans for which at least one covering chunk is in top-K
Doc Recall@K	Fraction of relevant documents among the first K distinct documents in the ranking

Why Evidence Recall is the primary recall metric. With overlapping chunks, one evidence span can be covered by two chunks. A system that retrieves just one of them has all the evidence but only 50% chunk recall. Evidence Recall instead asks whether each required piece of evidence was found; for a multi-hop item, whether *both* halves were retrieved. Unanswerable items have no evidence, so they are excluded from retrieval metrics and reported as excluded; generation evaluation scores them on abstention.

Uncertainty. Per-query metrics are averaged. We report 95% percentile-bootstrap intervals [13] (2,000 resamples of queries, fixed seed) for the headline metrics. With 198 queries overall, and only 13–42 per type, point estimates alone would overstate how precise these results are.

8 Baseline results

We evaluate dense retrieval, which is also the retrieval stage of the baseline RAG pipeline, on the test split. Setup:

- bge-small-en-v1.5 embeddings with the `Title > Section` prefix
- cosine HNSW index
- 198 answerable test items evaluated (21 unanswerable excluded)
- 0 unmapped evidence spans

The run is recorded as `20260928T080636-retrieval-dense-test` on dataset version `c6009ef85e53`, and a rerun reproduced every value exactly.

Table 11. Dense retrieval, test split (n = 198).

Metric	Value	95% bootstrap CI
Evidence Recall@5	0.701	0.641 – 0.759
Evidence Recall@10	0.761	0.710 – 0.815
MRR	0.624	0.573 – 0.682
NDCG@10	0.595	0.548 – 0.646
Chunk Recall@10	0.711	–
Doc Recall@5	0.838	–
Retrieval latency p50 / p95	24 ms / 32 ms	–

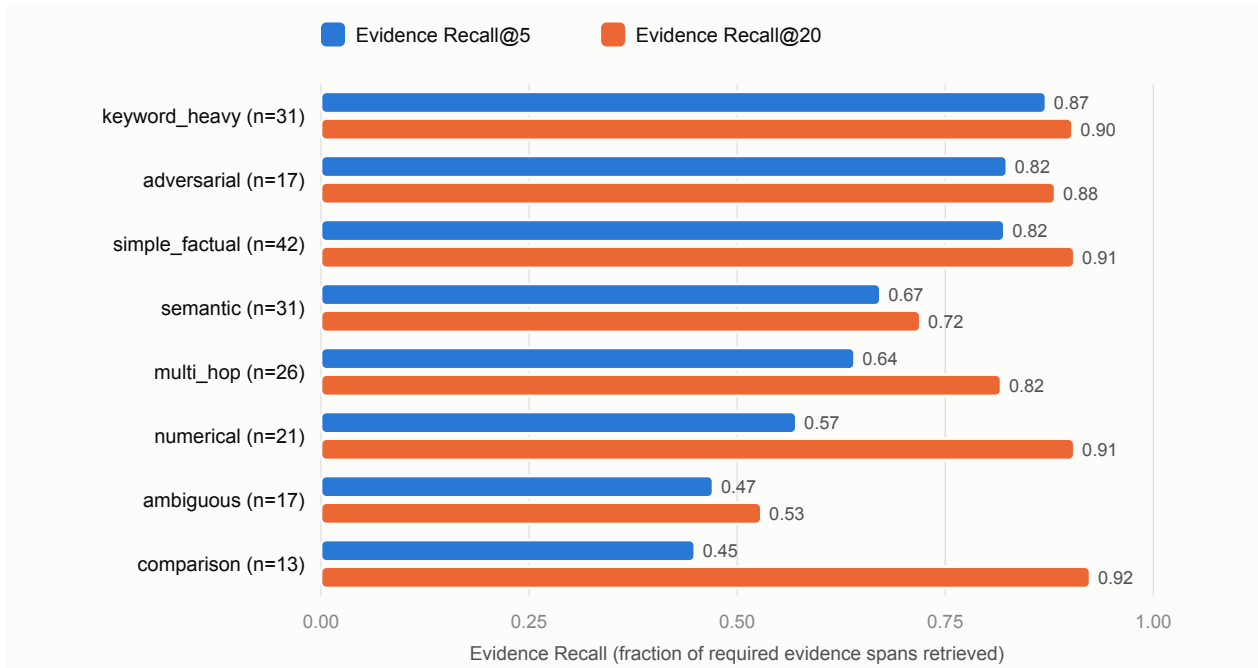


Figure 3. Evidence Recall@5 and @20 by question type (dense baseline, test split). The gap between the two bars shows how much evidence is retrieved but ranked below position 5. Values are also in Table 12.

Table 12. Evidence Recall@K and MRR by question type.

Type	n	@1	@5	@10	@20	MRR
keyword_heavy	31	0.565	0.871	0.871	0.903	0.707
adversarial	17	0.627	0.824	0.882	0.882	0.734
simple_factual	42	0.559	0.821	0.845	0.905	0.678
semantic	31	0.387	0.672	0.704	0.720	0.531
multi_hop	26	0.330	0.641	0.724	0.817	0.779
numerical	21	0.381	0.571	0.714	0.905	0.496
ambiguous	17	0.235	0.471	0.471	0.529	0.327
comparison	13	0.264	0.449	0.737	0.924	0.610
All	198	0.443	0.701	0.761	0.831	0.624

8.1 Discussion

Per-type samples are small, so these observations are directions for the next experiments rather than conclusions:

- **Ambiguous questions are hardest (0.47 at 5; 0.53 at 20).** Recall hardly improves with depth: the evidence is not being ranked low, it is not being retrieved at all. Reranking cannot fix this; query rewriting is the matching intervention.
- **Comparison (0.45 → 0.92) and numerical (0.57 → 0.91) gain most from 5 to 20.** The evidence is present in the candidate list but poorly ranked, which is where a cross-encoder reranker over a deeper candidate list should help.
- **Multi-hop has high MRR (0.78) but only 0.64 Evidence Recall@5.** One half of the evidence is found early and the other half often is not. MRR alone would suggest multi-hop is easy; Evidence Recall shows otherwise.
- **Semantic questions (0.67) trail simple factual ones (0.82),** as expected when the query avoids the passage's wording.
- **Keyword-heavy questions are already strong for dense retrieval (0.87).** Part of this is likely the lexical bias of passage-derived questions (§6.7). The BM25 experiment will show how much exact matching adds.

9 Limitations

- **Synthetic data without human validation.** The results measure retrieval against machine-generated, machine-validated labels. The sample audit estimates that a minority of items are flawed, but it is not a substitute for human review.
- **A single corpus and a single embedding model.** Conclusions about techniques may not transfer to other domains; larger embedders (e.g. 768-d) are an open experiment.
- **A small generator.** A 3B model limits both question quality and, in later phases, grading and judging quality. All generation metrics will be reported together with the model that produced them. The same experiments can be re-run with a stronger model by changing configuration only.
- **Code samples are not inlined.** `code_sample` shortcodes reference manifests outside the fetched subtree, so answers that exist only inside example YAML are absent from the corpus.
- **Only retrieval is evaluated so far.** Faithfulness, answer relevance, citation correctness, end-to-end latency and cost are not yet measured over the dataset.

10 Roadmap

The next phases add techniques one at a time and evaluate each on the same test split and metrics:

- BM25 (Phase 5)
- hybrid retrieval with weighted Reciprocal Rank Fusion (Phase 6)
- cross-encoder reranking (Phase 7)
- parent-child context expansion (Phase 8)
- query analysis and rewriting (Phase 9)
- corrective retrieval, with a small-model relevance grader (Phase 10)
- a Self-RAG-inspired loop with bounded iterations (Phase 11)
- claim-level verification (Phase 12)

Tracing and an experiment runner that records quality, latency, token usage and cost for each configuration follow. The generation evaluation will report deterministic signals (abstention, invalid citations) separately from judge-based scores, and will validate the judge against human labels before relying on it.

11 Reproducibility

```
docker compose up -d postgres # PostgreSQL 16 + pgvector 0.8.6, host port 5434
pip install -e ".[dev,local-llm]"
mlx_lm.server --model mlx-community/Qwen2.5-3B-Instruct-4bit --port 8080
python scripts/fetch_corpus.py # kubernetes/website @ 5dd61e1
EMBEDDING_DEVICE=mps python scripts/ingest.py --corpus kubernetes
python scripts/audit_chunk_tokens.py # Table 3
python scripts/create_eval_dataset.py # seed 42; top-up: --extend --seed 43
python scripts/run_retrieval_eval.py --retriever dense --split test # Tables 11-12
```

Table 13. Environment and artefacts.

Item	Value
Hardware	Apple M1, 8 GB RAM (development laptop)
Software	Python 3.12.6; FastAPI 0.141.1; SQLAlchemy 2.0.54; Pydantic 2.13.5; sentence-transformers 6.1.0; torch 2.14.0; mlx-lm 0.31.3; PostgreSQL 16.15; pgvector 0.8.6
Models	BAAI/bge-small-en-v1.5; mlx-community/Qwen2.5-3B-Instruct-4bit
Corpus	kubernetes/website 5dd61e17b222c0220281657b843a3e15c86b3cfe (docs v1.37), CC BY 4.0
Dataset	data/eval/kubernetes_v1.jsonl, version c6009ef85e53 (316 items; seeds 42, 43)
Code	Commits f2854e3 (Phase 1), 24659a9 (Phase 2), ab6ad5d (Phase 3), f46239c (Phase 4)
Key defaults	CHILD_CHUNK_SIZE 360, CHILD_CHUNK_OVERLAP 60, PARENT_CHUNK_SIZE 1600; cosine; TOP_K 10; LLM temperature 0; 144 tests passing

Documentation of the Kubernetes project is © The Kubernetes Authors and is used under CC BY 4.0.

12 References

- [1] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*.
- [2] Yan, S.-Q., Gu, J.-C., Zhu, Y., and Ling, Z.-H. (2024). Corrective Retrieval Augmented Generation. arXiv:2401.15884.
- [3] Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *ICLR*. arXiv:2310.11511.
- [4] Robertson, S. and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4).
- [5] Cormack, G. V., Clarke, C. L. A., and Büttcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. *SIGIR*.
- [6] Järvelin, K. and Kekäläinen, J. (2002). Cumulated Gain-Based Evaluation of IR Techniques. *ACM Transactions on Information Systems*, 20(4).
- [7] Malkov, Y. A. and Yashunin, D. A. (2020). Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE TPAMI*, 42(4).
- [8] Xiao, S., Liu, Z., Zhang, P., and Muennighoff, N. (2023). C-Pack: Packaged Resources To Advance General Chinese Embedding. arXiv:2309.07597.
- [9] Es, S., James, J., Espinosa-Anke, L., and Schockaert, S. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. arXiv:2309.15217.
- [10] Tang, L., Laban, P., and Durrett, G. (2024). MiniCheck: Efficient Fact-Checking of LLMs on Grounding Documents. *EMNLP*. arXiv:2404.10774.
- [11] Nogueira, R., Jiang, Z., Pradeep, R., and Lin, J. (2020). Document Ranking with a Pretrained Sequence-to-Sequence Model. *Findings of EMNLP*.
- [12] Qwen Team (2024). Qwen2.5 Technical Report. arXiv:2412.15115.
- [13] Efron, B. and Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.
- [14] Zheng, L., Chiang, W.-L., Sheng, Y., et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets and Benchmarks*.
- [15] Saad-Falcon, J., Khattab, O., Potts, C., and Zaharia, M. (2024). ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. *NAACL*. arXiv:2311.09476.
- [16] Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., and Gurevych, I. (2021). BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. *NeurIPS Datasets and Benchmarks*.